系统监视器（SysMon）

系 统 文 档

基于 Python + psutil + PySide6 的 Windows 系统性能监控程序

项目	内容
项目名称	系统监视器（SysMon）
项目类型	桌面系统监控工具（课程项目）
开发语言	Python 3.10+（验证环境 3.12.10）
GUI 框架	PySide6 6.12（Qt 6）
运行平台	Windows 10 / 11（64 位）
开源许可	MIT
代码库地址	https://github.com/wyhsjztd/psutil
文档版本	v1.0（2026-11-06）

2026 年 11 月

摘　要

本系统是一个运行于 Windows 平台的桌面性能监控程序，使用 Python 编写，采集层基于 psutil 与 Windows 性能计数器（PDH），界面层基于 PySide6（Qt 6）与 pyqtgraph 实现。程序参照 Windows 11 任务管理器「性能」页的布局，提供进程、CPU、内存、磁盘、网络、磁盘 I/O、服务、用户共八个功能页面，支持实时曲线、每逻辑处理器柱状图、进程搜索与排序、CSV 数据导出以及高占用告警。

系统采用「采集线程与 UI 线程分离」的架构：所有 psutil 调用均在独立线程完成，UI 线程只负责渲染，两者通过不可变的快照对象通信，从而保证一秒级采样下界面不卡顿。速率类指标由累计计数器差分得到，并在暂停、切换采样间隔时重建基线，避免虚假尖峰。CPU 利用率与磁盘活动时间直接读取 Windows 性能计数器，与任务管理器保持同一统计口径。

本文档记录该系统的 AI 工具使用情况、开发框架与技术、主要功能、系统运行截图以及用户操作手册，并附代码库地址。

目　录

1　项目概述.....5
1.1　项目背景.....5
1.2　主要目标.....5
1.3　系统特点.....5
2　系统运行截图.....5
2.1　进程页面（系统主界面）.....6
2.2　CPU 页面.....7
2.3　内存页面.....8
2.4　磁盘页面.....9
2.5　网络页面.....10
2.6　磁盘 I/O 页面.....11
2.7　服务页面.....12
2.8　用户页面.....13
3　所用的 AI 工具.....13
3.1　AI 参与的具体工作.....14
3.2　人工负责与验证.....14
4　开发框架与技术.....15
4.1　技术栈.....15
4.2　分层架构.....15
4.3　线程模型.....16
4.4　工程化.....17
5　实现的主要功能.....17
5.1　功能总览.....17
5.2　数据流与速率计算.....18
5.3　关键实现要点.....18

5.4　实测指标 19
6　用户操作手册 20
6.1　安装与启动 20
6.2　界面说明 20
6.3　常用操作 21
6.4　状态栏提示 22
6.5　常见问题 22
7　测试与构建 23
7.1　单元测试 23
7.2　打包为 exe 23
8　已知限制 24
附录 A　项目结构 24

1　项目概述

1.1　项目背景

Windows 自带的任务管理器能够直观地显示 CPU、内存、磁盘、网络的实时状态，但它不提供历史数据导出，也无法按自己的需要扩展指标。本项目以 psutil 为核心数据源，复刻任务管理器「性能」页的信息组织方式，并补充了数据导出、进程搜索、告警提示等实用功能，用于练习系统编程、多线程编程与桌面 GUI 开发。

1.2　主要目标

- 使用 psutil 采集 CPU、内存、磁盘、网络、进程、服务、用户等系统信息；
- 提供图形用户界面，布局与交互对标 Windows 任务管理器；
- 保证 1 秒采样下界面流畅，UI 线程不执行任何阻塞式系统调用；
- 关键指标与任务管理器口径一致，可直接对照核验；
- 代码结构清晰、核心算法可独立单元测试，并提供完整文档。

1.3　系统特点

- 八个功能页面，覆盖任务管理器「性能」页的全部主要指标；
- 深色界面，三层表面与悬停反馈，导航可切换状态一目了然；
- 采集全部在独立线程完成，实测 UI 单帧耗时 p95 由 6031 ms 降至 14 ms；
- 速率异常时显示「--」而非错误数值，宁可不显示也不显示错的；
- 可打包为单文件 exe（58.6 MB），双击即可运行，无需安装 Python。

2　系统运行截图

以下截图均取自程序在本机（Windows 11，14 核 20 逻辑处理器）上的实际运行画面，未做任何后期修饰。截图中的数值为采样瞬间的真实读数。

窗口由四个区域组成：顶部工具栏（采样间隔、暂停、导出 CSV）、左侧导航区（六个指标卡片与两个紧凑入口）、右侧页面区（当前页面的曲线与统计）、底部状态栏（最近采样时间、数据点数、告警与异常提示）。

2.1　进程页面（系统主界面）

进程页面是程序启动后的默认页面。左侧导航卡片显示六个指标的当前值与最近 60 秒迷你曲线；右侧为进程列表，包含名称、PID、CPU、内存四列，CPU 与内存列以半透明热度条辅助阅读，支持按名称或 PID 搜索、点击表头排序、右键结束任务。

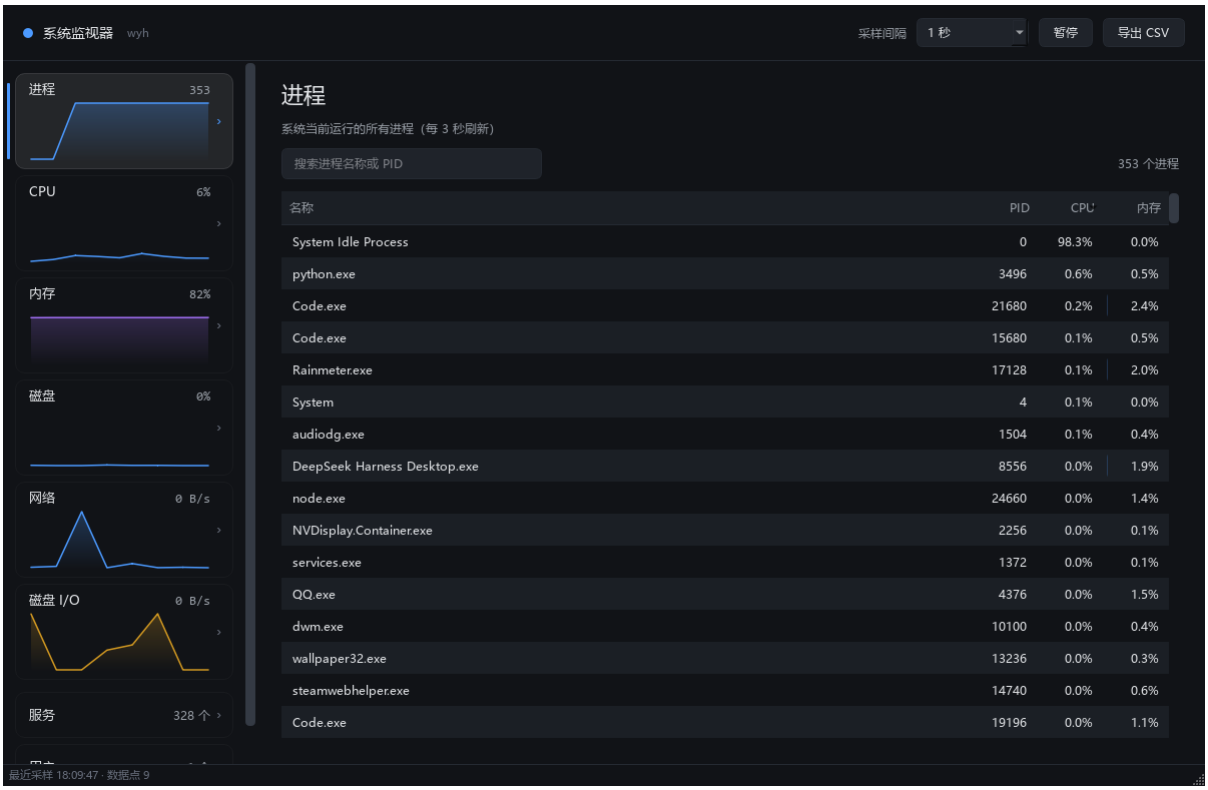


图 2-1　进程页面（系统主界面）

2.2 CPU 页面

CPU 页面显示总利用率实时曲线、每个逻辑处理器的柱状图、当前频率、进程数与正常运行时间。左上角大数字为当前总利用率，采样口径与任务管理器一致（频率加权）。

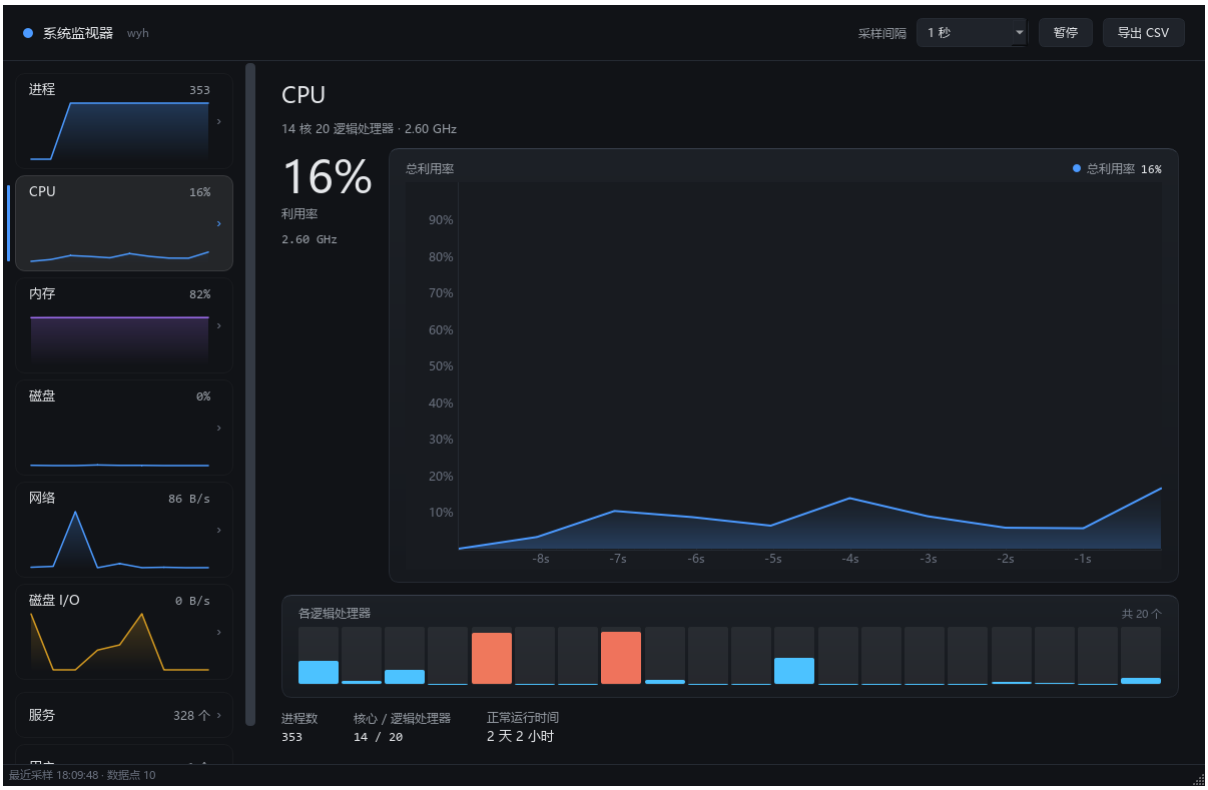


图 2-2　CPU 页面

2.3　内存页面

内存页面显示物理内存使用率曲线、已用 / 可用 / 总计容量、以及交换（分页）文件的使用情况。

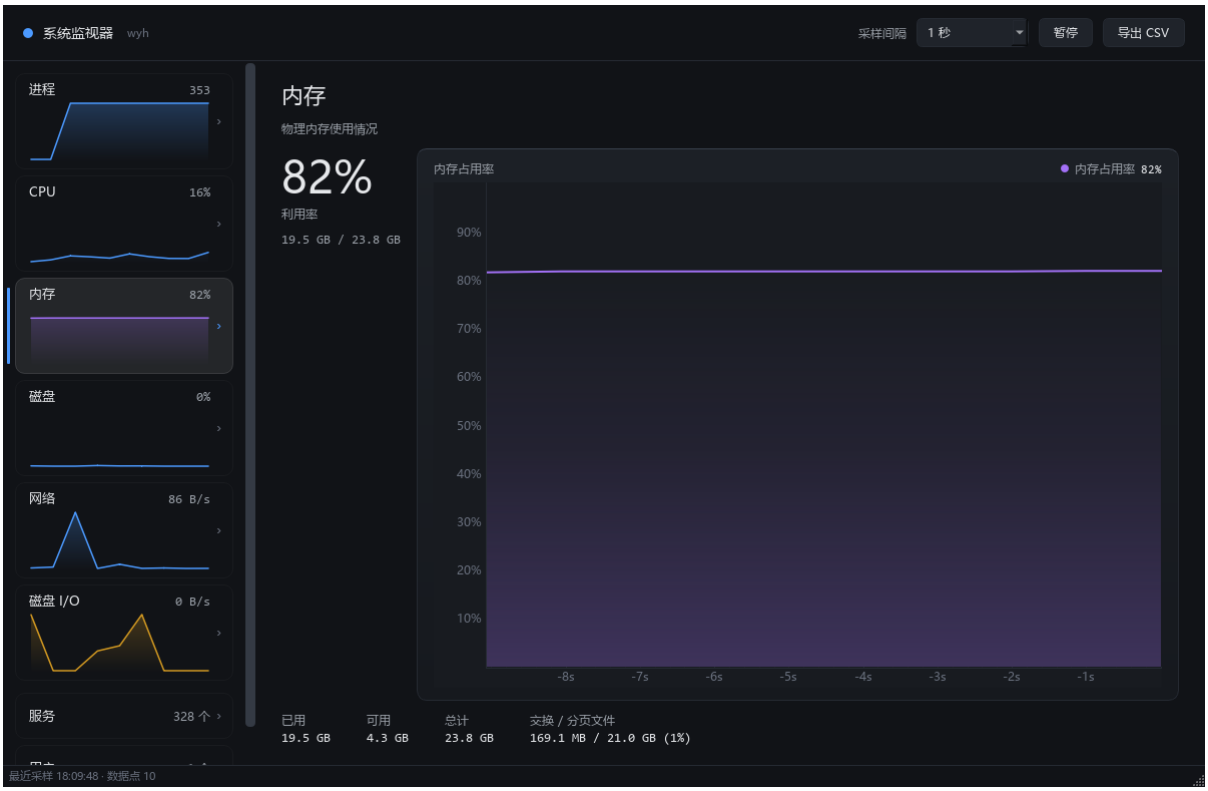


图 2-3　内存页面

2.4　磁盘页面

磁盘页面分为两部分：上方是各分区的容量使用率进度条与容量明细，下方是磁盘读写吞吐量曲线。页面明确标注了「容量使用率」与任务管理器「磁盘占用」（活动时间）的区别，避免混淆。



图 2-4　磁盘页面

2.5　网络页面

网络页面以双曲线显示接收与发送速率，下方汇总本次运行以来的累计流量与当前 TCP 连接数。



图 2-5　网络页面

2.6　磁盘 I/O 页面

磁盘 I/O 页面以双曲线显示读写速率，并统计累计读取量与写入量。该页面的单位是字节每秒（B/s），与磁盘页的百分比指标含义不同。



图 2-6　磁盘 I/O 页面

2.7　服务页面

服务页面列出本机全部 Windows 服务的名称、显示名称、状态与启动类型，运行中的服务以绿色标注，支持按名称或显示名称搜索。服务枚举通过 WMI 完成，在独立线程中每 30 秒刷新一次。

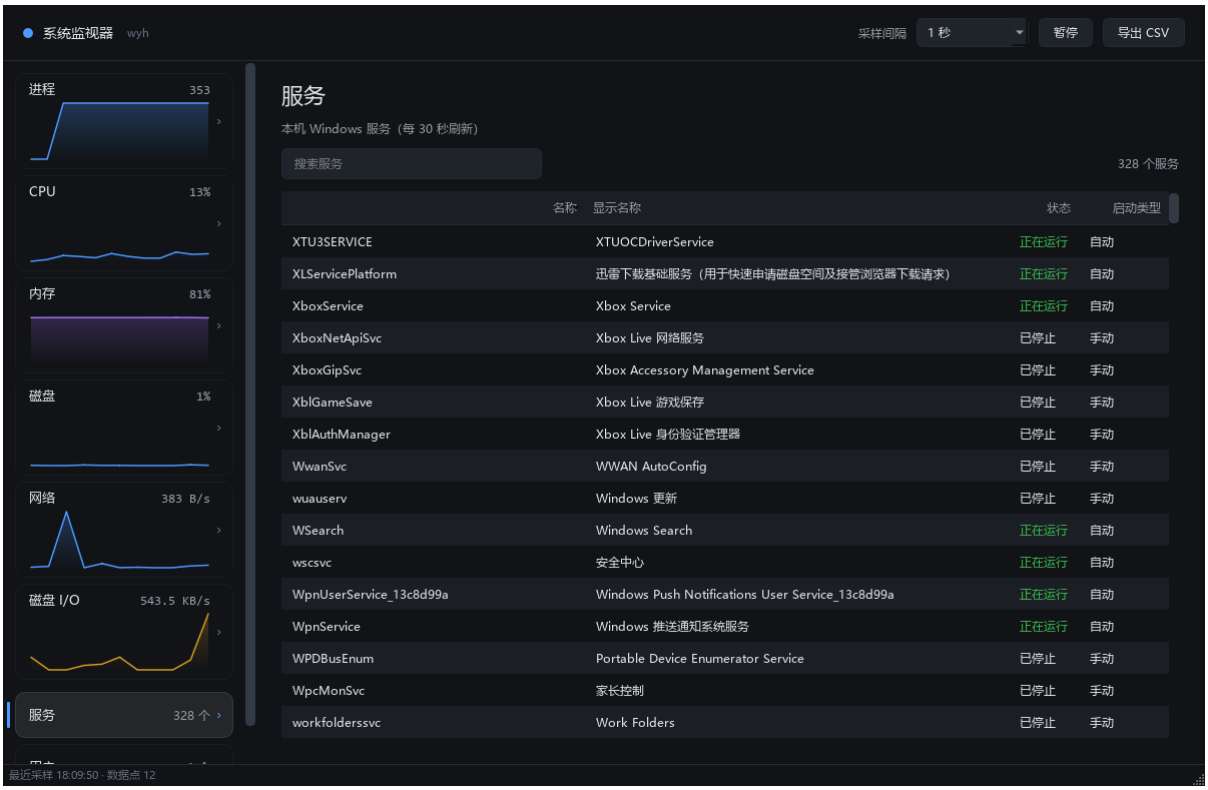


图 2-7　服务页面

2.8 用户页面

用户页面列出当前登录到本机的用户、主机、会话与登录时间。若某项信息系统未提供，则显示「--」。

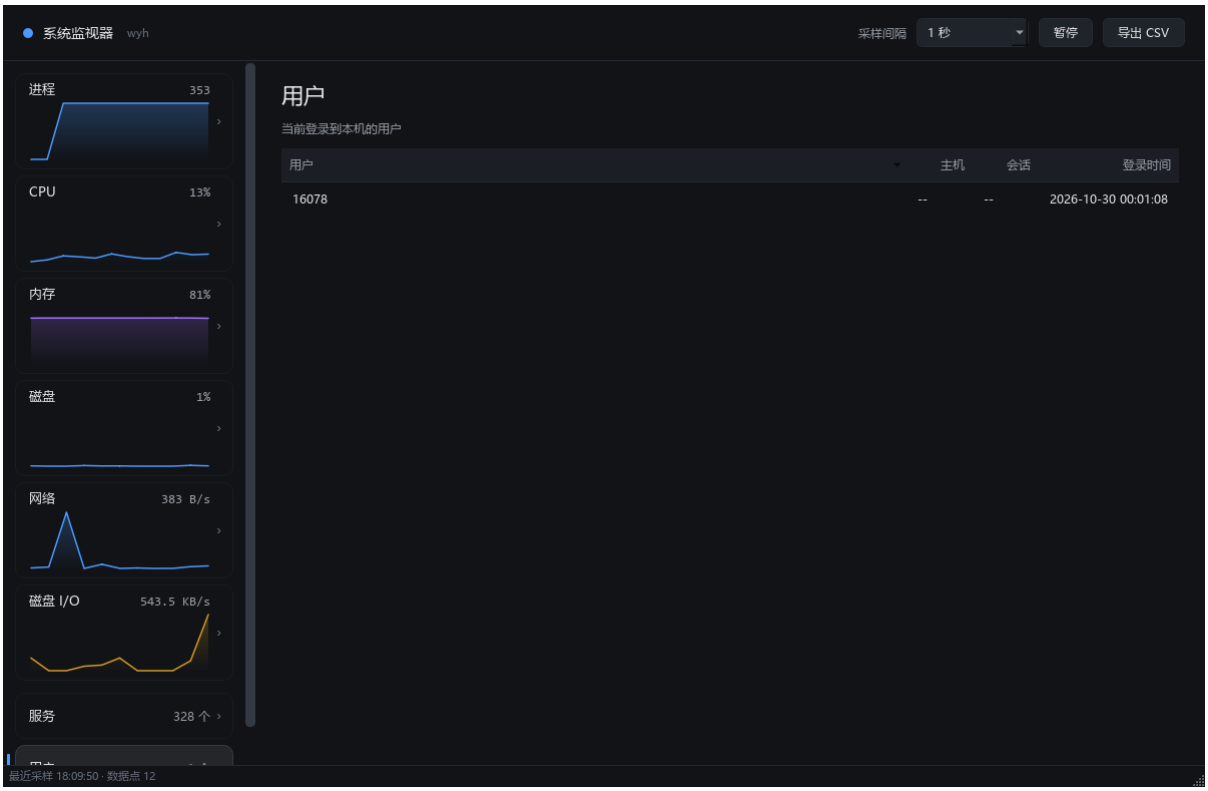


图 2-8　用户页面

3 所用的 AI 工具

本项目在开发过程中使用了 AI 编程助手，具体配置如下。

项目	说明
助手框架	Claude Code（Anthropic 的命令行 AI 编程助手）
底层大模型	DeepSeek 大模型
使用方式	在项目目录中通读代码库、生成与修改源码、定位缺陷、撰写文档、执行构建与验证

3.1 AI 参与的具体工作

1. 需求拆解与技术选型：确定 psutil 采集 + PySide6 渲染 + pyqtgraph 绘图的组合，划分纯逻辑层与 UI 层；
2. 代码生成：数据模型、采集器、采样线程、八个功能页面、深色样式表、打包脚本；
3. 缺陷定位：进程表卡顿（单帧 6031 ms）、速率长期显示「--」、CPU 读数与任务管理器不符、磁盘百分比语义混淆、导航卡片悬停无反馈等；
4. 文档撰写：设计文档、用户手册与本系统文档；
5. 构建与验证：在干净虚拟环境中打包 exe，用脚本测量启动耗时、自身 CPU 占用与退出残留。

3.2 人工负责与验证

AI 生成的内容并非直接采用，以下环节由人工完成：

- 提出需求、验收结果，并在真实机器上对照 Windows 任务管理器逐项核对数据；
- 最终取舍：例如拒绝毛玻璃、页面切换动画等效果，理由见设计文档第 6.1 节；
- 所有关键结论均经实测复核。

一个典型例子：AI 最初用 GetSystemTimes 自证「CPU 读数没有问题」，但该系统调用与 psutil 使用的是同一种统计口径（% Processor Time），两者互相印证只能得到同一个错误结论。直到把程序读数与任务管理器逐帧比对，才发现任务管理器显示的是频率加权的 % Processor Utility，两者在 CPU 睿频时相差数倍。这个问题最终通过改用 PDH 计数器解决。

这说明 AI 给出的结论存在「看起来正确」的风险，因此本项目对每一条关键读数都做了与系统工具的对照验证，未经验证的内容不写入文档。

4 开发框架与技术

4.1 技术栈

技术	版本（已验证）	用途	选择理由
Python	3.12.10	运行环境	psutil 原生支持，开发效率高
psutil	7.2.2	采集 CPU / 内存 / 磁盘 / 网络 / 进程 / 用户	跨平台系统信息库，接口稳定、覆盖面全
PySide6	6.12.0	图形用户界面	Qt 官方 Python 绑定，信号槽机制天然隔离采集线程与 UI 线程
pyqtgraph	0.14.0	实时曲线绘制	基于 Qt，自带坐标轴与交互，绘制 360 个点开销极低
numpy	2.4.4	曲线数据组织	pyqtgraph 的数据载体，NaN 可直接表达曲线断点
ctypes + PDH	标准库 / Windows API	读取系统性能计数器	任务管理器的 CPU、磁盘百分比来自 PDH，psutil 无法直接取得
unittest	标准库	单元测试	无需额外依赖，一条命令即可运行
PyInstaller	6.x	打包单文件 exe	免安装 Python 即可运行，便于交付

为什么选择 PySide6 而不是 Tkinter：Tkinter + matplotlib 在 1 Hz、60 个点的场景下完全可用，本项目选择 PySide6 的原因不在性能，而在于 QThread 与信号槽提供了语言级的跨线程通信保障、pyqtgraph 与 Qt 布局体系一致、QSS 可低成本实现统一的深色主题。需要说明的是，pyqtgraph 默认使用 QPainter 光栅渲染而非 OpenGL，本项目也未开启 useOpenGL，以避免在虚拟机、远程桌面等环境下出现驱动问题。

4.2 分层架构

系统按「采集 / 逻辑 / 渲染」三层划分，核心约束是：UI 线程不调用 psutil，采集线程不触碰控件，二者只通过 Signal(object) 传递不可变的快照对象。

```
UI 线程
  MainWindow（顶部工具栏 / 左侧导航 / 8 个页面 / 状态栏）
        ▲  Signal(object) 快照
采集线程
  SampleWorker  → Collector → psutil + PDH 性能计数器
  ProcessWorker → Collector.refresh_processes()
  ServiceWorker → psutil.win_service_iter()
逻辑层（纯 Python，无 psutil / Qt 依赖，可离线单测）
  engine：速率差分 / 历史窗口 / 告警滞回 / 格式化
```

模块	职责	依赖
models.py	冻结数据类，定义跨线程传输的快照结构	无
engine.py	速率差分、历史窗口、告警滞回、格式化	无（可单测）
collector.py	调用 psutil，隔离异常，缓存分区列表	psutil
winperf.py	通过 ctypes 调用 PDH 性能计数器	ctypes
worker.py	采样线程与进程采集线程	Qt
services.py	Windows 服务枚举线程	Qt + psutil
ui/*	主题、控件、曲线、页面、主窗口	Qt

4.3 线程模型

线程	周期	内容	停止方式
UI 线程	—	渲染与交互	—
SampleWorker	1 秒（可调）	性能指标采集	可中断 Event.wait + wait()
ProcessWorker	3 秒 / 30 秒	全量进程遍历	同上，另用事件立即唤醒
ServiceWorker	30 秒	服务列表枚举	同上

进程遍历在 Windows 上约需 1 秒，必须独立成线程：若放在 1 Hz 主采样线程内，实际间隔会被拉长到 5 秒左右，网络与磁盘速率会因间隔异常而长期显示「--」。

4.4 工程化

- 测试：tests/test_engine.py 共 19 个用例，覆盖差分、首帧、时间异常、计数器回绕、基线重置、告警滞回与格式化；
- 打包：build_exe.bat 自动创建干净虚拟环境后打包，产物为单文件、无控制台窗口的 exe；
- 版本控制：Git 管理，源码与文档托管于 GitHub。

5 实现的主要功能

5.1 功能总览

页面	功能
进程	全部进程列表（名称 / PID / CPU / 内存），实时搜索、表头排序、CPU 与内存热度条、右键结束任务
CPU	总利用率曲线、每逻辑处理器柱状图、当前频率、进程数、正常运行时间、高占用告警
内存	使用率曲线、已用 / 可用 / 总计、交换（分页）文件
磁盘	磁盘活动时间占比、各分区容量使用率与容量明细、读写吞吐曲线
网络	接收 / 发送速率双曲线、累计流量、TCP 连接数
磁盘 I/O	读 / 写速率双曲线、累计读写量
服务	Windows 服务列表（名称 / 显示名称 / 状态 / 启动类型），支持搜索
用户	当前登录用户、主机、会话与登录时间

通用能力：

- 采样间隔 1 / 2 / 5 秒可切换；支持暂停与继续；窗口最小化时自动暂停采集；
- 历史缓存 1 小时，可一键导出 CSV（UTF-8 with BOM，Excel 直接打开不乱码）；
- 速率异常（首帧、暂停恢复、改间隔、计数器回绕）显示「--」，不产生错误数值；
- CPU 超过 80% 触发状态栏告警，降至 75% 解除，两次提示间隔至少 5 分钟。

5.2 数据流与速率计算

```
psutil / PDH  →  Collector  →  Snapshot（冻结）  →  Signal  →  MainWindow
                                       │
                                       └→  History(3600)  →  曲线数据 / CSV 导出
```

网络与磁盘 I/O 的 psutil 返回值是自开机以来的累计计数器，必须对两次采样做差分：

```
rate = (curr_value - prev_value) / (curr_time - prev_time)
```

以下四种情况返回 None，界面显示「--」：首帧；时间差小于等于 0；实际间隔偏离名义间隔超过 max(2 倍名义间隔, 0.5 秒)；计数器回绕（差值为负）。设计原则是宁可不显示，也不显示错误的数值。

暂停恢复、切换采样间隔时，程序会显式重建基线，并清掉 psutil 内部的缓存（cpu_percent 的模块级上次时间、process_iter 的进程对象缓存），否则恢复后的第一帧会给出跨越暂停区间的平均值。

5.3 关键实现要点

5.3.1 CPU 口径与任务管理器对齐

psutil 的 cpu_percent() 返回 % Processor Time（纯时间占比），而任务管理器显示的是 % Processor Utility（按当前频率加权）。CPU 处于睿频或降频状态时两者相差数倍。开发机实测同一时刻的读数如下：

来源	读数
\Processor(_Total)\% Processor Time（psutil 使用）	1%
\Processor Information(_Total)\% Processor Utility	16%
Windows 任务管理器显示	21%

因此程序改用 PDH 的 % Processor Utility，总量与每核心保持一致；psutil 的 cpu_percent() 仅作为 PDH 不可用时的回退方案。

5.3.2 磁盘的两个百分比区分开

指标	含义	典型值
磁盘活动时间（导航卡片、磁盘页）	区间内有 I/O 在途的时间占比，与任务管理器「磁盘」同义	空闲 0–5%
分区容量使用率（磁盘页分区列表）	已用空间 / 总空间，反映磁盘装得多满	与负载无关

活动时间同样取自 PDH 的 \PhysicalDisk(_Total)\% Disk Time。psutil 的 read_time / write_time 在 NVMe 或未启用 diskperf 的机器上恒为 0（实测写入 213 MB 后增量仍为 0），只作回退。初版只提供容量使用率，用户拿它和任务管理器对比时误以为读数出错（任务管理器 5%，程序 88%），因此现在把两个指标分列并明确标注。

5.3.3 进程表性能

初版使用 QTableWidget，每 3 秒重建 340 行 × 4 列的单元格对象，单次重建实测最长 6031 毫秒，界面直接卡死。改为 QTableView + QAbstractTableModel + QSortFilterProxyModel 后，刷新只做一次 beginResetModel / endResetModel，耗时降至 13 毫秒；UI 线程单帧 p95 由 6031 毫秒降至 14 毫秒，事件循环吞吐由 8.7k 提升到 115k 帧每秒。

5.3.4 进程 CPU 口径归一化

psutil 的 Process.cpu_percent() 以单个核心为 100%，在多核机器上可超过 100%。本程序按逻辑处理器数归一化，使「全部处理器合计 = 100%」，与任务管理器可直接对照。

5.4 实测指标

指标	数值
exe 体积	58.6 MB（55.9 MiB，单文件）
冷启动耗时	约 4 秒（解压运行库到临时目录）
程序自身 CPU 占用（稳定态）	0.39%
内存占用	约 170 MB
关闭后残留进程	0

指标	数值
单元测试	19 个，全部通过

6　用户操作手册

6.1　安装与启动

6.1.1　直接运行 exe（推荐）

双击 dist\SysMon.exe 即可启动，不需要安装 Python，也不会弹出命令行窗口。单文件约 58.6 MB，首次启动约 3–4 秒（需先把运行库解压到临时目录）；程序图标已内置，关闭窗口后不会残留后台进程。若被杀毒软件拦截，请将 exe 加入信任列表，这是 PyInstaller 打包程序的常见误报。

6.1.2　从源码运行

```
git clone https://github.com/wyhsjztd/psutil.git
cd psutil

python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

python main.py        # 或双击 run.bat（pythonw 启动，无控制台窗口）
```

国内网络下载缓慢时，可使用清华镜像：pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple

环境要求：Windows 10 / 11、Python 3.10 及以上、约 300 MB 磁盘空间（主要来自 PySide6）。

6.2　界面说明

程序启动后，窗口分为四个区域：

- 顶部工具栏：显示程序名称与本机主机名，右侧为「采样间隔」下拉框、「暂停」按钮、「导出 CSV」按钮；
- 左侧导航区：六个指标卡片（进程、CPU、内存、磁盘、网络、磁盘 I/O）显示当前值与最近 60 秒迷你曲线，下方为「服务」「用户」两个紧凑入口；

- 右侧页面区：当前选中页面的曲线与统计数据；
- 底部状态栏：最近采样时间、数据点数、告警与采集异常提示。

左侧导航项在鼠标悬停时会提亮并显示「›」箭头，单击即可切换页面。

6.3 常用操作

6.3.1 修改采样间隔

点击顶部「采样间隔」下拉框，可选 1 秒 / 2 秒 / 5 秒。切换后程序会自动重建速率基线，因此不会出现「间隔变了、算出一个巨大尖峰」的问题；曲线上新旧采样密度的点按各自的时间位置连续显示，不会中断。历史数据仍保留在内存中，可完整导出。

6.3.2 暂停与继续

点击「暂停」停止采样，按钮变为「继续」，界面保持最后一帧数据。点击「继续」后，程序丢弃旧的速率基线并重新起算，第一帧速率显示「--」，随后恢复正常；暂停期间的曲线显示为空白，而不是被一条直线连接。

6.3.3 导出数据

点击「导出 CSV」，选择保存位置即可。导出范围为本次运行期间的全部采样点（最多 1 小时）；文件编码为 UTF-8 with BOM，中文 Windows 上可直接用 Excel 双击打开；表头包含单位，时间列为 ISO 8601 格式。

导出列为：时间、CPU(%)、内存(%)、磁盘活动(%)、分区最大容量使用率(%)、网络下行(B/s)、网络上行(B/s)、磁盘读(B/s)、磁盘写(B/s)、进程数。

6.3.4 结束进程

在「进程」页右键单击任意进程，选择「结束任务：进程名（PID）」，确认后即可结束。结束系统关键进程可能导致系统不稳定，请谨慎操作；若提示「权限不足」，请以管理员身份重新启动程序。

6.3.5 搜索与排序

搜索：在进程页或服务页顶部输入框输入关键字，列表实时过滤。排序：单击表头可在升序与降序之间切换，数值列按数值大小排序而非按字符串。

6.3.6 最小化

窗口最小化时自动暂停采样，还原后自动恢复，避免在后台白白占用 CPU。

6.4 状态栏提示

提示	含义
正在建立采样基线	刚启动、暂停恢复或切换间隔，速率尚未就绪，约 1 秒后恢复
⚠ CPU 占用率过高	CPU 连续超过 80%，降至 75% 解除，两次提示间隔至少 5 分钟
采集异常：磁盘 / 网络	该指标本次读取失败，其余指标不受影响
已导出 N 条记录 → 路径	CSV 导出成功

6.5 常见问题

6.5.1 网络速率显示「--」？

程序刚启动、暂停恢复后，或系统卡顿导致采样间隔异常时，速率需要重建基线，约 1 秒后自动恢复。

6.5.2 数值与任务管理器不一致？

- CPU：本程序与任务管理器同为「频率加权」口径，可直接对照；perfmon 等工具默认显示「时间占比」，睿频时会低很多，属正常现象；
- 磁盘：注意三个指标的区别——导航卡片与磁盘页的「活动时间」对应任务管理器的磁盘百分比；分区列表的百分比是「容量使用率」（装得多满），与系统忙闲无关；磁盘 I/O 页是吞吐量（B/s）；
- 网络：任务管理器可能以 Mbps 显示，本程序以 B/s（字节）显示，需乘以 8 换算；
- 进程 CPU：本程序为 3 秒区间平均值，任务管理器为瞬时值，允许存在小幅差异。

6.5.3 磁盘页缺少某个分区？

光驱、已拔出的移动硬盘、网络映射盘会被自动过滤，避免访问无效设备导致界面卡死。

6.5.4 服务页一直显示「正在读取…」？

服务枚举通过 WMI 完成，首次加载可能需要数秒，之后每 30 秒刷新一次。

6.5.5 关闭窗口后进程没有退出？

正常关闭即退出。若出现异常，可在任务管理器中结束对应进程。程序的采样线程使用可中断等待，正常退出不会残留。

6.5.6 如何卸载？

exe 版直接删除 exe 即可，程序不写注册表、不在用户目录留残留；源码版删除项目目录与虚拟环境即可。若曾添加杀毒软件信任项，请一并移除。

7 测试与构建

7.1 单元测试

核心速率算法与告警逻辑与 psutil、Qt 完全解耦，可单独测试。运行方式：

```
python -m unittest discover -s tests -v
```

共 19 个用例，覆盖正常差分、首帧、时间差小于等于 0、间隔异常、计数器回绕、RateTracker 重置、告警滞回与冷却、字节格式化边界。实测全部通过。

7.2 打包为 exe

```
build_exe.bat
```

脚本会自动创建 .venv-build 干净虚拟环境再打包。这一步不能省：PyInstaller 会沿依赖链扫描，若直接用开发机的全局环境，很容易把其他项目的重型依赖一并收进 exe。实测全局环境打包时把 torch、cv2、scipy 等约 1 GB 的依赖打了进去，产物从 90 MB 膨胀到 444 MB；改用最小依赖集后产物为 58.6 MB。

8　已知限制

1. 仅在 Windows 11 上验证；服务页依赖 win_service_iter，其他平台为空。
2. 网络速率为所有网卡汇总值，包含回环、VPN 与虚拟机虚拟网卡。
3. 磁盘活动时间依赖 PDH 计数器，若系统禁用了相应计数器则显示「--」。
4. 进程 CPU 为 3 秒采样区间内的平均值，与任务管理器的瞬时值存在差异。
5. 服务列表每 30 秒刷新一次，状态变化存在延迟。
6. CPU 利用率采用频率加权口径，与任务管理器一致，但会高于性能监视器的 % Processor Time；跨工具对比时需确认对方使用的计数器。
7. 不持久化历史数据，程序关闭后清空。
8. 采样精度受系统调度影响，高负载时实际间隔可能略大于设定值。

附录 A　项目结构

```
psutil/
├── main.py                  入口
├── sysmon/
│   ├── models.py            冻结数据模型
│   ├── engine.py            速率计算 / 历史窗口 / 告警（纯逻辑）
│   ├── collector.py         psutil 采集封装
│   ├── winperf.py           PDH 性能计数器（ctypes）
│   ├── worker.py            采样线程
│   ├── services.py          服务采集线程
│   └── ui/
│       ├── theme.py         配色与 QSS
│       ├── widgets.py       导航卡片 / 迷你曲线 / 分区行
│       ├── charts.py        pyqtgraph 曲线卡片
│       ├── pages.py         八个功能页面
│       └── main_window.py   主窗口
├── tests/test_engine.py
├── docs/                    设计文档 / 用户手册 / 系统文档 / 截图
├── assets/
├── requirements.txt
├── requirements-build.txt
├── build_exe.bat
├── run.bat
└── LICENSE
```